

Week 5: Programming Parametric Engineering Models

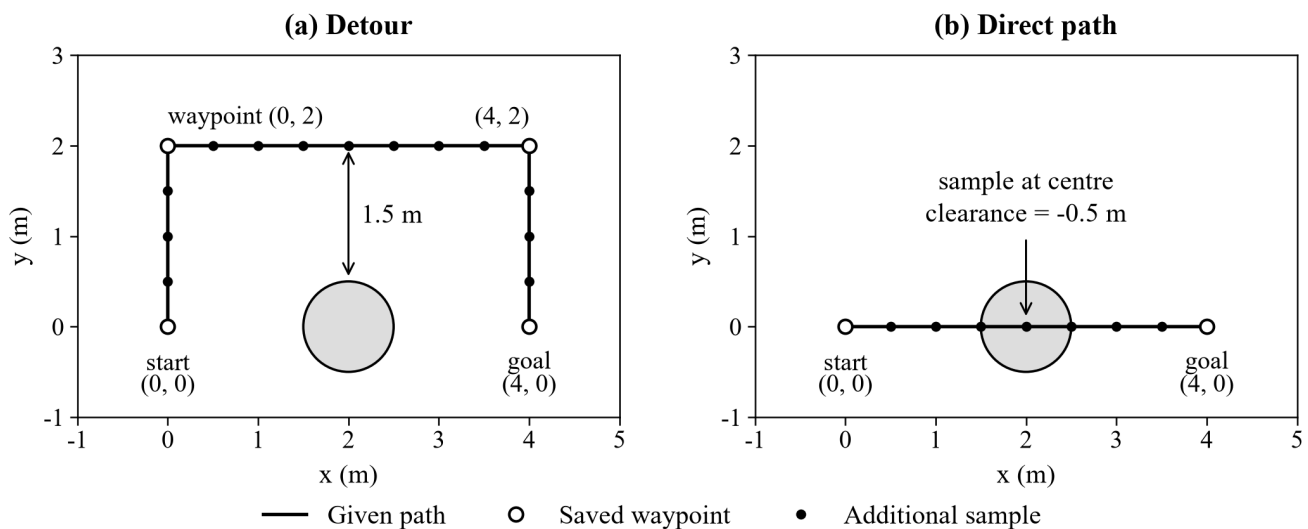
Build a reusable program that evaluates a supplied path

School of Mechanical Engineering, Soongsil University
2026-2 Applied Programming

A path may pass a check because it avoids an obstacle, because we allowed it to come closer, or because we checked too few places to notice a problem. This week we build a program that measures a supplied path and compares those measurements with stated limits. Its result includes the measurements, how far each is from its limit, and whether both conditions pass. The path and the functions that calculate its geometry are already provided.

1 Represent the problem and identify what may change

Imagine a point moving on a flat map. The coordinates x and y are in metres. Its start is $(0, 0)$ and its goal is $(4, 0)$. A disk-shaped obstacle has centre $(2, 0)$ and radius 0.5 m. The workspace allows $-1 \leq x \leq 5$ and $-1 \leq y \leq 3$. Figure 5.1 shows the supplied detour and a direct path through the same scene.



Fixed obstacle: centre $(2, 0)$ m, radius 0.5 m.

Interpolation spacing: 0.5 m. Both panels use the same workspace and obstacle.

Figure 5.1: The fixed scene and two supplied paths. Axes are in metres. Open circles are saved positions. Filled dots are additional positions where the program calculates distance to the obstacle. The direct path's endpoints are outside the disk, although part of its segment is inside. The distance arrow measures clearance to the disk boundary.

A saved position is a **configuration**. Here it represents a point position, not a robot's joint angles or the space occupied by a physical robot. Each saved position on a path is a **waypoint**. We are given the ordered path

$$(0,0) \rightarrow (0,2) \rightarrow (4,2) \rightarrow (4,0).$$

The order matters because consecutive waypoints define straight segments. The two middle points describe the detour above the disk. The direct path will help us explain a limitation of checking only saved positions.

The provided `Configuration`, `ConfigurationBounds` and `Path` classes keep the definitions used in Week 4. The `Obstacle` class keeps the same ID, centre and radius meanings. A `Path` stores an ordered tuple of `Configuration` objects. `path.start` and `path.goal` read its endpoint objects. The expression `configuration.dimension` reads a property, the number of coordinates. The parentheses in `path.length()` request that the object's length method run and return a number. Requesting this execution is a **call**. The starter supplies completed versions of these objects in its own `ap_week05` package. Use this week's imports rather than mixing class imports from different weekly packages. You do not need to copy answers from a previous assignment.

Week 1 used calculated values and limits to return an accepted/not-accepted decision. Week 2 taught functions, interpolation and tests. Week 3 introduced objects and composition. Week 4 organized those objects as collections and saved scene data. We reuse these programming tools here.

The new step is to explain *why* a decision changes. A signed margin tells us how much a condition passes or fails by. A sampling method determines which positions were actually checked. Comparing one change at a time lets us tell a different path from a different rule for accepting it.

We will compare three situations: the same path under different length limits, the same path checked at different positions, and two paths judged by the same rules. The evaluation program returns measurements and margins as well as a Boolean. Code that uses its result can then see what a True result means: which points were checked and which limits they passed. We do not search for a new route or implement the supplied geometry algorithms.

1.1 Prepare the examples once

The following setup is the same weekly workflow used earlier in the course. Prepare it once now. The examples in Section 2 use completed support functions, so they run before you complete any Assignment TODO. Use the Week 5 template link announced through LMS. Follow the same GitHub account and authentication procedure used in Weeks 1–4. Select **Use this template**, then **Create a new repository**. Under your own account, create a **Private** repository named `applied-programming-w05-YOUR-STUDENT-ID`. Replace `YOUR-STUDENT-ID` with your student ID. Use this same Week 5 repository throughout the assignment. Do not rename or overwrite an earlier week's repository.

In your new repository, use the **Code** button to copy its HTTPS clone URL. Open **Anaconda Prompt** on Windows, not Python's `>>>` prompt. The folder and account below are replacement values, not commands to copy unchanged.

Run – your course parent folder

```
cd /d "C:\your-course-folder"
git clone YOUR-PRIVATE-HTTPS-CLONE-URL
cd applied-programming-w05-YOUR-STUDENT-ID
git remote -v
```

The two **origin** addresses must point to your private Week 5 repository. If Git asks you to sign in, use the previously taught Git Credential Manager procedure as the repository owner. Never put a password or token in the URL.

Keep the established Conda and pip sequence. The supplied `environment.yml` creates `applied-programming-w05` with Python 3.12.13 and pip 26.2.1. The requirements file installs NumPy 2.4.6, pytest 9.1.1 and setuptools 80.9.0. This separate weekly environment preserves the earlier weeks' installed packages. No graphics library, GPU, simulator or new environment manager is required.

Run once – Anaconda Prompt, repository root

If the Week 5 environment already exists, skip creation and continue with activation. Do not delete an existing environment to repeat this command.

```
set PYTHONUTF8=1
conda --no-plugins env create --solver classic --file environment.yml
```

Activate, install and check – same folder

```
conda activate applied-programming-w05
python -m pip install -r requirements.txt
python -m pip install -e . --no-build-isolation
python scripts/verify_environment.py
```

Expected status

```
ENVIRONMENT PASS
```

The editable installation makes saved source edits available on the next run. The environment check confirms the required interpreter, installed dependencies and package import. It does not check the unfinished model. If it fails, read the reported mismatch before continuing. Initial installation needs an internet connection. Core calculations and tests then run locally without network access.

When reopening Anaconda Prompt, change back to the repository root, run `set PYTHONUTF8=1`, and activate `applied-programming-w05` with the same Conda activation command above. Do not recreate the environment after each TODO. The Windows `set` command enables UTF-8 in the current prompt. Use the Week 2 folder syntax in a Conda-initialized macOS/Linux terminal and omit that Windows-only line. These instructions have been verified on Windows. The macOS/Linux procedure has not been separately verified for this week.

Which supplied files will we use?

Before the first example, locate the following files in your repository. They are already complete and are available for reading:

- `examples/compare_evaluations.py` runs the measurements and comparisons in Sections 2 and 4. Its last command word selects what to run.
- `examples/inspect_path.py` shows one calculation from beginning to end. Section 4 uses a practice copy to change a length limit.
- In `src/ap_week05/`, `domain.py` defines the supplied objects and `cases.py` builds the one scene used here. `provided.py` contains `sample_path`, `path_metrics` and `constraint_margins`.

The two example scripts use these completed functions directly, so they can run while the Assignment methods in `src/ap_week05/model.py` are still unfinished. Keep the supplied files unchanged. The Assignment identifies `model.py` and `tests/test_student.py` as the two files you will edit. The code excerpts below are parts of the existing files, not additional programs to create or functions to implement.

1.2 Parameters, a method choice and fixed values

We need names for the values that control evaluation before we can write a useful model. **Clearance** means distance from a checked point to the obstacle's boundary. A required clearance of 0.5 m asks the point to remain at least that far outside the disk. A maximum path length of 9 m limits the total travel distance.

Checking a path also requires a choice about *where* to measure clearance. A **sample point** is a position on the path where the program calculates distance to the obstacle. For example, between (0, 0) and (4, 0), the program can calculate this distance at (0, 0), (0.5, 0), and so on through (4, 0). These nine positions are the samples. They do not replace the path's two saved endpoints. The **waypoints** method checks only the saved waypoints. The **interpolated** method adds check points along each straight segment. **Interpolation** here means calculating intermediate positions between two known endpoints. It does not find a route around the obstacle. The **resolution**, in metres, is the largest allowed distance between successive samples on a segment. A smaller value usually means more checks. In Week 2, `interpolate_segment(start, goal, num_samples)` used a requested *number of points* for one segment. This week's supplied `sample_path` applies the same idea of straight-line interpolation to a whole path, but takes a *maximum spacing* instead. It determines how many points each segment needs. It is a new function with different inputs, not a renamed copy of the Week 2 function.

A **parametric model** uses named values to describe a problem while its calculation rules remain the same. The following table separates the input roles in our model. The accepted numerical ranges are course-defined limits for this small exercise, not universal engineering limits.

Setting and example	Role	Accepted values
<code>min_clearance_m</code> Example: 0.5 m	Acceptance limit Required distance outside the disk.	$0 \leq c_{\min} \leq 2 \text{ m}$
<code>max_length_m</code> Example: 9.0 m	Acceptance limit Greatest acceptable path length.	$0 < L_{\max} \leq 20 \text{ m}$
<code>resolution_m</code> Example: 0.5 m	Checking resolution Maximum spacing between interpolated samples.	$0.05 \leq h \leq 1 \text{ m}$
<code>method</code> Example: "interpolated"	Checking method Selects which supplied procedure runs.	"waypoints" or "interpolated"

The first three settings are continuous quantities stored with finite precision. The method is a discrete choice. Use the last column to validate an input, not to infer whether an evaluated path will pass its constraints.

The **fixed environment** consists of the workspace bounds, disk centre and radius. The **supplied candidate** is the ordered path and its endpoints. These describe what we evaluate, rather than how we check it.

A continuous setting can vary by an amount, such as 0.5 to 0.6 m. A discrete choice selects one named alternative, not a value between the names. Keep the environment fixed when comparing settings. Moving the disk changes the problem. Relaxing an acceptance limit or checking fewer points can change the reported decision without improving the route. The `_m` suffix means metres: it does not convert millimetres automatically.

2 Calculate measurements before making a decision

2.1 Length, clearance and turning

With the inputs named, the next task is to measure the supplied path. A **metric** is a numerical description of a result. Our supplied `path_metrics` function returns three named measurements. Students call this function. They do not implement the geometry inside it.

For waypoints $\mathbf{q}_0, \dots, \mathbf{q}_{n-1}$ in metres, the path length L , also in metres, is

$$L = \sum_{i=0}^{n-2} \|\mathbf{q}_{i+1} - \mathbf{q}_i\|_2.$$

Here n is the number of waypoints and $\|\cdot\|_2$ is ordinary straight-line distance. In Figure 5.1(a), the three segment lengths are 2, 4 and 2 m. Their sum is 8 m. The saved path geometry determines this length. Changing the checking resolution does not change that path.

For a position $\mathbf{s}$ where we calculate distance, obstacle centre $\mathbf{o}$ and radius r , the supplied point-clearance calculation is

$$d(\mathbf{s}) = \|\mathbf{s} - \mathbf{o}\|_2 - r.$$

All terms are in metres. The program reports the minimum of $d(\mathbf{s})$ over the samples it checked. The result is positive outside the disk, zero on its boundary and negative inside it. In the detour with 0.5 m interpolation spacing, the smallest value is $2 - 0.5 = 1.5$ m. It occurs, for example, at (2, 2). This value describes distance to the disk, not to the workspace boundary.

The third measurement, `total_turn_rad`, adds the absolute change of direction at each interior waypoint. A straight continuation contributes zero. The detour turns through a right angle at each of its two interior waypoints, so its total is $\pi/2 + \pi/2 = \pi$ radians, approximately 3.141593. A radian is the angle unit used by the supplied calculation. For reference, a right angle is $\pi/2$ radians. This course uses total turning as a simple geometric **smoothness metric**: less turning means a straighter path by this measure. It says nothing about speed, acceleration or how comfortably a physical machine could follow the path. It is a reported metric in this assignment, not an additional pass/fail constraint. Now read how the supplied comparison file obtains these numbers. Open `examples/compare_evaluations.py`. Its `measurements` function requests the positions from `sample_path`, then passes those positions to `path_metrics`. These two functions are imported from the completed `ap_week05` package.

Code – supplied `examples/compare_evaluations.py`, measurement function

```
def measurements(case, path, method="interpolated", resolution_m=0.5):
    samples = sample_path(path, resolution_m=resolution_m, method=method)
    metrics = path_metrics(path, samples, case["obstacle"])
    return metrics, len(samples)
```

The `def` line defines the function and names its inputs. It does not yet run the body. If no other choices are supplied, `method="interpolated"` and `resolution_m=0.5` give the defaults. In `samples = sample_path(...)`, the expression on the right calls the function with a path and these choices. The function returns an array. The `=` assigns that returned array to the name `samples`. The call on the next line passes the path, that array and the obstacle to `path_metrics`, whose returned dictionary receives the name `metrics`. `return metrics, len(samples)` returns two values: the measurements and the number of rows in the sample array. It does not print either value.

The following parts of the same file prepare the detour and use that function. The supplied `supplied_case()` function builds the scene in Figure 5.1 and returns a dictionary. Its keys name the start, goal, bounds, obstacle and path objects. For example, `case["path"]` selects the four-waypoint detour, and `case["obstacle"]` selects the disk centred at (2, 0) with radius 0.5 m.

Code – same file, preparation and metrics option

```
case = supplied_case()
detour = case["path"]

metrics, count = measurements(case, detour)
for name, value in metrics.items():
    print(f"{name}: {value:.6f}")
print(f"sample_count: {count}")
```

For this input, `samples` has shape (17, 2): 17 positions, with an x and a y coordinate in each row. Its first two rows are (0, 0) and (0, 0.5), and its final row is (4, 0). The returned `metrics` dictionary contains "length_m": 8.0, "clearance_m": 1.5 and "total_turn_rad" approximately 3.141593. The assignment `metrics, count = measurements(case, detour)` receives the two returned values in order, so `count` becomes 17.

The loop reads each dictionary key into `name` and its number into `value`. The `print` calls write those values to the terminal. Inside the formatted string, `:.6f` displays six digits after the decimal point. It changes the displayed text, not the stored measurement. Run the following command from the repository root. The final word, `metrics`, selects this part of the script. The other choices `method`, `resolution`, `limits` and `paths` select comparisons we will read next. They are arguments to this script, not Python commands.

Run – activated Week 5 environment, repository root

```
python examples/compare_evaluations.py metrics
```

Expected output

```
length_m: 8.000000
clearance_m: 1.500000
total_turn_rad: 3.141593
sample_count: 17
```

The first three lines confirm the hand calculations. `sample_count: 17` reports the number of positions where distance was calculated. The next section explains why that count matters. Keep the supplied file unchanged. If a command fails, read its last error line and check the active environment and repository folder. An import error is not an expected unfinished-model failure.

2.2 What the checking method changes

The `sample_path` function creates the positions used for clearance checks. In `waypoints` mode, the detour has four samples. In `interpolated` mode with resolution 0.5 m, the segments require 4, 8 and 4 intervals. There are $4 + 8 + 4 + 1 = 17$ sample points, because shared segment endpoints are counted only once. The `sample_path` function chooses enough equal intervals that each interval is no longer than the requested resolution. The spacing need not equal that value exactly.

Figure 5.1(b) gives a reason to care about this choice. Its direct path saves only (0, 0) and (4, 0). Waypoint checking sees two points that are each 1.5 m outside the disk. Interpolation at 0.5 m also checks (2, 0), which is the disk centre. Its clearance is -0.5 m. Both computations follow their selected rules, but they examine different points. Neither method changes the original two-endpoint path.

The following run holds the direct path and scene fixed and changes only the method. It prints the measurements before applying acceptance limits. The script first builds `direct` from the existing start and goal objects. Its loop then requests measurements twice, once with each method name:

Code – same file, direct path and method option

```
direct = Path([case["start"], case["goal"]])

for method in ("waypoints", "interpolated"):
    metrics, count = measurements(case, direct, method=method)
```

Both calls still use the same `measurements` function. In `method=method`, the left name identifies the function's parameter and the right name reads the loop's current string. No new value for `resolution_m` is given, so the default 0.5 m remains. The rest of the existing branch prints one table row for each result. The final command word `method` selects this comparison.

Run – repository root

```
python examples/compare_evaluations.py method
```

Expected output

```
path=direct; resolution_m=0.500000
method      sample_count length_m clearance_m
waypoints   2           4.000000 1.500000
interpolated 9           4.000000 -0.500000
```

Both rows have length 4 m. The different clearance values describe different sets of checked positions on that same path. The path did not move away from the disk when the program checked only its endpoints.

Resolution can also change the checked positions without changing the path. Keep the method interpolated and compare 1.0 m with 0.5 m spacing. The `resolution` option changes the number passed as `resolution_m`:

Code – same file, resolution option

```
for resolution in (1.0, 0.5):
    metrics, count = measurements(case, direct, resolution_m=resolution)
```

Here the loop variable `resolution` is first 1.0, then 0.5. The omitted `method` argument keeps the default "interpolated".

Run – repository root

```
python examples/compare_evaluations.py resolution
```

Expected output

```
path=direct; method=interpolated
resolution_m sample_count length_m clearance_m
1.000000     5           4.000000 -0.500000
0.500000     9           4.000000 -0.500000
```

The count grows from five to nine, but clearance stays -0.5 m because both sets already include the disk centre. More samples need not change the reported minimum. For other spacings, `sample_path` redistributes the sample points, so a finer grid need not contain every point from a coarser grid. Do not assume its reported minimum must decrease on every refinement.

This is a limitation of **sampled checking**. An unchecked part of a segment may violate a condition. Finer checking can find failures missed by coarser checking, but no result here is a certificate of physical safety. When comparing

candidate paths later, keep the checking conditions fixed: use the same scene, checking method, resolution and acceptance limits. Choosing fewer checks merely to obtain a passing result is not an improvement to the route. `resolution_m` is still validated in waypoint mode, although that mode does not use it to insert samples.

2.3 Constraint margins explain the Boolean result

The measurements do not yet state whether a path is acceptable. A **constraint** is a condition the path must satisfy. Our two operational conditions are $C \geq c_{\min}$ and $L \leq L_{\max}$, where C is the sampled minimum clearance, $c_{\min}$ the required clearance, L the measured length and $L_{\max}$ its limit.

A **constraint margin** reports how much room remains before the limit is violated. Choose the subtraction direction so an acceptable value gives a nonnegative result:

$$m_C = C - c_{\min}, \quad m_L = L_{\max} - L.$$

Both margins have unit metres. For clearance, more is allowed, so subtract the minimum from the measurement. For length, less is allowed, so subtract the measurement from the maximum. A positive margin leaves room, zero lies exactly on the accepted boundary, and a negative margin identifies a violated condition.

For the detour, $L = 8$ m and $C = 1.5$ m. Keep the minimum clearance at 0.5 m, so $m_C = 1.0$ m throughout. Change only the maximum length: 9, 8, then 7 m. The length margin becomes 1, 0, then -1 m. The middle case lies exactly on the *length* boundary. Its clearance still has room to spare. The supplied `constraint_margins` function performs these subtractions. The program combines the signs to produce the Boolean `feasible`. It is true only when *both* margins are at least zero.

The next function in `examples/compare_evaluations.py` connects the measurements to that decision:

Code – same file, margin and decision function

```
def margins_and_decision(metrics, max_length_m=9.0):
    margins = constraint_margins(
        metrics["length_m"], metrics["clearance_m"],
        min_clearance_m=0.5, max_length_m=max_length_m,
    )
    feasible = all(value >= 0.0 for value in margins.values())
    return margins, feasible
```

The expressions `metrics["length_m"]` and `metrics["clearance_m"]` read the two measured numbers from the dictionary. `constraint_margins` returns a new dictionary containing the two subtractions, which `margins` receives. `margins.values()` gives those two numbers. The expression `value >= 0.0` tests each one, and `all(...)` is True only when both tests are true. The function returns the dictionary and that Boolean. With the default limit 9 m, they are margins 1 m and 1 m, and True.

The `limits` option measures the detour once and then changes only the maximum length passed to this function:

Code – same file, limits option

```
metrics, _ = measurements(case, detour)

for maximum in (9.0, 8.0, 7.0):
    margins, feasible = margins_and_decision(metrics, max_length_m=maximum)
```

The name `_` receives the sample count, which this comparison does not use. Each loop iteration uses the same `metrics`. Only `maximum` changes, so the table below must keep the measured length and clearance unchanged.

Run – repository root

```
python examples/compare_evaluations.py limits
```

Expected output

```
path=detour; method=interpolated; resolution_m=0.500000
min_clearance_m=0.500000
max_length_m length_m clearance_m length_margin_m feasible
9.000000      8.000000 1.500000      1.000000      True
8.000000      8.000000 1.500000      0.000000      True
7.000000      8.000000 1.500000     -1.000000     False
```

The three rows describe one unchanged path, measured in one unchanged way. Only its permitted length changes. A **False** becoming **True** after relaxing that limit would mean the requirement changed, not that the path became shorter. This is why a result should retain its measurements and signed margins.

One satisfied constraint cannot compensate for another failed one. Keep the signed values in the result so code using that result can identify the failed condition. Do not clip a negative margin to zero or round a value before checking its sign. Formatting a number to six decimal places is for display only. An equality boundary is included in the contract. Numerical tests may use a tolerance to compare calculated measurements, but that test tolerance does not silently relax the model's ≥ 0.0 decision rule.

Throughout this chapter, **feasible means satisfying the selected sampled checks and the stated length limit**. Workspace membership is checked before evaluation, as described next. Feasible does not mean an unsampled continuous path or a physical robot has been verified safe.

3 Give valid inputs and evaluated results different meanings

3.1 Reject an undefined request, report a failed constraint

The margin calculation assumes that the input describes a meaningful problem. A negative interpolation spacing or a string such as "half a metre" cannot be used as the required numeric setting. A coordinate containing `float("nan")` is not a finite position. These are **invalid inputs**. The program raises the existing `PlanningModelError`, a specific kind of `ValueError`. Code requesting the calculation receives this error because its inputs do not describe an allowed calculation.

By contrast, a valid input can fail an evaluated constraint. The direct path in Figure 5.1(b), checked in `interpolated` mode at 0.5 m resolution, has a negative clearance margin and returns `feasible=False`. It is **infeasible** under that check, not malformed. Waypoint-only checking can miss the violation, as Section 2.2 showed. Return the `EvaluationResult`, not an exception, when the selected checks find a negative margin.

The distinction determines *when* each check happens. As in Week 3, `__post_init__` rejects inconsistent inputs when the dataclass is created. The settings must have accepted types and ranges. The connected objects must describe one two-dimensional path whose endpoints and workspace membership agree with the request. Type annotations do not enforce these conditions. The exact construction checks are collected once in Assignment TODOs 1–2.

One difference from Week 2 needs a reason. When we measured only path length, two equal consecutive positions contributed a length of zero and were allowed. This evaluation also measures changes of direction. A zero-length segment has no direction, so the evaluation problem rejects consecutive equal positions before calculating turns. The general `Path` class is unchanged and can still store them. The extra restriction belongs to `ParametricPlanningProblem` and the supplied evaluation functions. The two-dimensional scene and the exercise's limit of 2–32 waypoints are also requirements of this evaluation problem, not a new definition of every `Path`.

After construction succeeds, `evaluate()` calculates the margins even when one is negative. For example, changing `max_length_m` from 9 to 7 is a valid request that returns a failed length condition. Changing it to `-1` is outside the allowed input range in Section 1.2 and raises an error instead. Do not treat these two outcomes as interchangeable.

3.2 Compose the model from supplied objects

The earlier `PlanningProblem` expressed a request using start, goal and bounds. Here we already have a candidate path and need to judge it against an obstacle and selected limits. The new `PlanningSettings` dataclass groups the four settings from Section 1.2. The new `ParametricPlanningProblem` dataclass connects start, goal, bounds, obstacle, path and settings. It is a separate class, not a renamed version or subclass of `PlanningProblem`. Its `evaluate()` method uses those stored objects to request the supplied calculations and return their result. This uses composition from Week 3: one object connects other objects so they can be used together. The geometry functions are already complete.

The data moves through a specific sequence:

inputs $\rightarrow$ samples $\rightarrow$ measurements $\rightarrow$ margins $\rightarrow$ decision.

For the detour, this means four saved waypoints become 17 checked positions, the measurements include $L = 8$ m and $C = 1.5$ m, both default margins are 1 m, and the decision is `True`. The samples determine where clearance is measured. The limits affect the margins, not the saved waypoints or measured length. Section 2 showed the supplied functions that perform these steps. Section 4 will connect their returned dictionaries to the model's result and to the code that displays it.

Both model classes use `@dataclass(frozen=True)`. This prevents normal reassignment of their fields. It does not magically freeze a mutable list inside an object. The supplied configuration/path classes store owned tuples, and their array conversion methods return new arrays. Evaluation must not change its path, settings or fixed scene.

The new `EvaluationResult` records the numbers and decision from this evaluation. It has a different meaning from the earlier `PlanResult`. The earlier `succeeded` property reports whether a result has a path. It does not establish that the path passes this week's two limits. Here `feasible` is true only when the clearance and length margins are both nonnegative under the selected checking method.

The following table identifies the fields in the supplied result dataclass. Use it when constructing or inspecting the value returned by `evaluate()`. Code using this object can read and test its numbers before choosing how to display or store them.

Field	Meaning and example
<code>length_m</code>	8.0 m for the detour.
<code>clearance_m</code>	1.5 m under the stated checking method.
<code>total_turn_rad</code>	π radians for its two right-angle turns.
<code>clearance_margin_m</code>	1.0 m under the default clearance limit.
<code>length_margin_m</code>	1.0 m under the default length limit.
<code>feasible</code>	Python Boolean True under these two nonnegative margins.
<code>method, sample_count</code>	"interpolated" and 17, identifying the check used.

Read the measurements together with the margins, method and sample count. For example, `length_m` describes the path, whereas `length_margin_m` compares that length with the chosen limit. `sample_count` tells how many positions support the reported clearance. The result contains these values, not a picture.

3.3 Prepare display data without depending on a display

Drawing the path needs coordinate lists. Deciding whether the path passes its checks needs numbers. Keep the two responsibilities separate. The already completed `to_plot_data()` method returns fresh plain lists for `path_xy`, `samples_xy`, `obstacle_center_xy` and `labels`, plus the scalar `obstacle_radius_m`. You use this supplied method unchanged. The provided `PlotData` type annotation describes these dictionary entries. It is not an additional class to implement. For the detour, `path_xy[1]` is `[0.0, 2.0]`. These lists can be read by a supplied drawing function without changing the path objects.

Each use of `to_plot_data()` returns new coordinate lists, so editing a display list does not move the model's path. This reuses the ownership rule from Week 3. The supplied implementation and public tests already cover it. There is no additional ownership method or student test to write.

The `evaluate()` method never runs the drawing function. It returns its `EvaluationResult` even if no window can open. An optional supplied writer can save the display data as an SVG, a picture that a browser can open. A supplied 3D renderer can also replay the path and show the checked points after the model is complete. Both use the same model data. Drawing or animating a picture does not validate a path, and a failed picture write does not undo a calculated result.

4 Connect comparisons to a reusable evaluation program

4.1 Compare paths with the same checking conditions

We have changed limits and checking choices while keeping a path fixed. Now change the path while keeping the rules fixed. Use the same disk and bounds, interpolated checks at 0.5 m, minimum clearance 0.5 m and maximum length 9 m for both candidates.

The final option, `paths`, changes the path sent to the same measurement function. It leaves the default method, resolution and limits unchanged:

Code – same file, paths option

```
for name, path in (("direct", direct), ("detour", detour)):
    metrics, _ = measurements(case, path)
    margins, feasible = margins_and_decision(metrics)
```

The loop first selects `direct`, then `detour`. The corresponding name is only the printed row label. The first function returns that path's measurements. The second returns its margins and decision. The remaining table-formatting code displays one row per path.

Run – repository root

```
python examples/compare_evaluations.py paths
```

Expected output

```
method=interpolated; resolution_m=0.500000
min_clearance_m=0.500000; max_length_m=9.000000
path  length_m clearance_m clearance_margin_m length_margin_m feasible
direct 4.000000 -0.500000 -1.000000 5.000000 False
detour 8.000000 1.500000 1.000000 1.000000 True
```

The direct path is shorter, but its clearance margin is -1 m. The detour is longer, yet both its margins are positive. Under these unchanged conditions, it is the one that satisfies both conditions. This comparison does not claim that the detour is the best possible route or that a physical robot is safe.

These runs support three different explanations. Changing a limit changes what we accept. Changing the method or resolution changes what we inspect. Changing the path changes the candidate being judged. A useful comparison states which of these changed and keeps the other relevant inputs fixed.

4.2 Follow one result and save a boundary case for testing

Open the existing `examples/inspect_path.py`. It runs the same three calculations for one detour, without selecting a comparison option. Its first part obtains `case`, `samples` and `metrics` in the order we traced in Section 2.1. At its default settings, the positions have shape `(17, 2)`, the length is 8 m and the minimum clearance is 1.5 m. We will change one limit in a practice copy and retain that example for a test.

Before doing so, use the following table to locate the inputs and returned values needed by `evaluate()`. The left column gives *function names*. A call is an expression such as `path_metrics(case["path"], samples, case["obstacle"])` that supplies their inputs and requests execution. All three functions are complete in `src/ap_week05/provided.py`, so you use them without rewriting their calculations.

Supplied function	Inputs and returned values
<code>sample_path</code>	Accepts <code>path</code> , <code>resolution_m</code> , <code>method</code> . Returns a new NumPy array with two columns, one row per checked position.
<code>path_metrics</code>	Accepts <code>path</code> , <code>samples</code> , <code>obstacle</code> . Returns a dictionary with <code>length_m</code> , <code>clearance_m</code> , <code>total_turn_rad</code> .
<code>constraint_margins</code>	Accepts <code>length_m</code> , <code>clearance_m</code> , <code>min_clearance_m</code> , <code>max_length_m</code> . Returns a dictionary with <code>clearance_margin_m</code> and <code>length_margin_m</code> .

Use these supplied functions in row order. The positions returned by `sample_path` become an input to `path_metrics`. Its length and clearance then become inputs to `constraint_margins`. The keyword names match

the model and result fields. This table lets you check, for example, that `constraint_margins` receives measured numbers and acceptance limits, not the path object itself.

The remaining part of `inspect_path.py` calculates margins and prints the results. The constants at the top of that file are `MIN_CLEARANCE_M = 0.5` and `MAX_LENGTH_M = 9.0`.

Code – supplied examples/inspect_path.py, margins and display

```

margins = constraint_margins(
    metrics["length_m"], metrics["clearance_m"],
    MIN_CLEARANCE_M, MAX_LENGTH_M,
)
for name, value in metrics.items():
    print(f"{name}: {value:.6f}")
for name, value in margins.items():
    print(f"{name}: {value:.6f}")
print("feasible:", all(value >= 0.0 for value in margins.values()))
print("sample_count:", len(samples))

```

The first loop prints the three numbers in `metrics`. The next loop prints the two numbers in `margins`. The final two lines display whether both margins pass and how many positions were measured. With the original constants, the margins are $1.5 - 0.5 = 1.0$ m and $9.0 - 8.0 = 1.0$ m, so the Boolean printed is `True`. These `print` statements display values that have already been calculated. They are separate from the functions that return those values. This example still uses no unfinished model method.

Create a practice copy once, or reopen it if it already exists. Do not overwrite an existing practice file. Keep the supplied example unchanged.

Run once – repository root

```
copy examples\inspect_path.py practice\inspect_path.py
```

In `practice/inspect_path.py`, keep `MIN_CLEARANCE_M = 0.5` and set only `MAX_LENGTH_M = 8.0`. Leave the path, method and resolution unchanged. Save the file and run it:

Run – repository root

```
python practice/inspect_path.py
```

Expected output

```

length_m: 8.000000
clearance_m: 1.500000
total_turn_rad: 3.141593
clearance_margin_m: 1.000000
length_margin_m: 0.000000
feasible: True
sample_count: 17

```

Only the length margin is zero. It is accepted. Now change the same constant to `MAX_LENGTH_M = 7.999`, save, and rerun the same command:

Expected output

```
length_m: 8.000000
clearance_m: 1.500000
total_turn_rad: 3.141593
clearance_margin_m: 1.000000
length_margin_m: -0.001000
feasible: False
sample_count: 17
```

The measured length and clearance stay unchanged, while the length margin is -0.001 m. The decision becomes **False**. Save this practice file and use the two limit values and their expected results for the Assignment's boundary test. The file is ungraded and not an additional submission.

The supplied demo in `src/ap_week05/__main__.py` shows how another part of a program will use your completed model. It constructs settings and a problem, requests an evaluation, and prints a value from the returned object. This is the code that *calls* the model, also called its **caller**. Unlike the examples above, it runs only after model TODOs 1–3 are complete.

Code – supplied `src/ap_week05/__main__.py` (caller excerpt)

```
case = supplied_case()
problem = ParametricPlanningProblem(
    start=case["start"],
    goal=case["goal"],
    bounds=case["bounds"],
    obstacle=case["obstacle"],
    path=case["path"],
    settings=PlanningSettings(),
)
result = problem.evaluate()
print(f"length_m: {result.length_m:.6f}")
```

The expression `PlanningSettings()` constructs the default settings object. The enclosing `ParametricPlanningProblem(...)` call uses it together with the named input objects to construct `problem`. Then `problem.evaluate()` requests the calculation from that particular problem and returns an `EvaluationResult` object. In `result = problem.evaluate()`, the `=` gives that object the name `result`. It does not print the object. The expression `result.length_m` reads its stored length, 8.0. Only the final `print` writes `length_m: 8.000000` to the terminal. The complete demo prints the remaining fields in the same way. The Assignment will run this demo with `python -m ap_week05` after the model is complete. Tests can inspect the returned object's numbers directly, without extracting them from displayed text.

5 Week 5 Assignment: Evaluate Supplied Paths

5.1 Purpose and work to complete

Complete a program that checks whether a supplied path meets the stated clearance and length limits. The expression `problem.evaluate()` asks the problem object to run this calculation. It must return an `EvaluationResult` containing the measurements, signed margins and a `True` or `False` decision. Use the supplied functions to compare paths with the same obstacle, limits and checking settings. Then change one length limit, or change only the positions at which distance is calculated, and explain why the decision changes. You do not write path-search or geometry algorithms. **The complete Assignment works without visualization.**

Files you edit

Paths start at the repository root, the folder containing `README.md`.

File	Your task
<code>src/ap_week05/model.py</code>	Complete TODO 1–3
<code>tests/test_student.py</code>	Write tests for three behaviors

Several TODOs belong to one source file. There is no separate file to create for each method. Keep class names, fields, signatures, the result type and the supplied `to_plot_data()` method. Replace TODO 4 and the skipped placeholder in the test file with your own tests for the three behaviors specified below. The object definitions in `domain.py`, input data in `cases.py`, and calculations in `provided.py` are complete files in `src/ap_week05/`. The scripts in `examples/` are also complete. Your model connects those existing pieces. Keep them, the optional drawing code, environment files, public tests and supplied case unchanged. They may be read to understand a failure, but are not student edit targets.

Complete and check

1. Prepare your private Week 5 repository and environment.
2. Implement each model TODO, save, and run its focused tests.
3. Write your three behavior checks, run the full tests and inspect the demo result.
4. Push the two edited files and submit the revision through LMS.

The implementation checkpoint is **71 public tests passing**. Your own tests must cover **all three required behaviors**. With the three independent tests in the reference example, the complete suite has **74 passes**. Your total may differ if a test is parametrized or you add tests. Every required behavior must be covered without failures or the skipped placeholder. Passing tests is not the same as submitting through GitHub and LMS.

5.2 Prepare the repository and environment

Reuse the private `applied-programming-w05-YOUR-STUDENT-ID` repository and core environment prepared in Section 1.1. If you have not prepared them, complete that section through its successful environment check before continuing. Do not create another repository or repeat setup just because the Assignment section begins here. Complete the supplied comparisons and saved practice in Sections 2 and 4 before editing the model. In a new Anaconda Prompt, return to the repository root and activate `applied-programming-w05` as shown in Section 1.1. Send the SSUMechE invitation at LMS submission as described in Section 5.5.

Run – untouched starter, repository root

```
python -m pytest tests/public tests/test_student.py -q
```

Expected output (summary)

```
69 failed, 2 passed, 1 skipped
```

The TODO bodies intentionally raise `NotImplementedError`. These failures mean the model is unfinished. Two tests of the supplied functions already pass, and the student-test placeholder is skipped. Replace that placeholder in TODO 4. Import errors, missing packages and a failed environment check are not this expected baseline. Resolve those setup errors first.

The core environment is enough for every required task. Optional SVG and 3D views are collected in Section 5.6. You may use them after completing TODO 3, without waiting until submission.

5.3 Implement TODO 1–3

For each TODO, open the named method in `src/ap_week05/model.py`, replace its unfinished body, save the file and run the focused check shown below. Keep the existing class names, fields and method signatures.

TODO 1 – Validate PlanningSettings

Edit: `PlanningSettings.__post_init__`. Use the existing imports and the settings in Section 1.2. For each numeric setting, follow this validation order:

1. Accept only ordinary Python `int` or `float`, not `bool`.
2. Check the full allowed range, such as `0.0 <= value <= 2.0`. Zero clearance is allowed, but zero maximum length is not.
3. Convert an accepted value with `float` and check `math.isfinite`.
4. Store the accepted float. For example, 9 becomes 9.0. Inside this frozen dataclass's initializer, use `object.__setattr__(self, "max_length_m", float(value))`, substituting the corresponding field name. The supplied configuration uses the same technique.

Also accept only the two stated method names. Raise `PlanningModelError` whenever a check fails. A `raise` statement stops construction with that error. Returning `False` from `__post_init__` does not reject the object.

Expected: valid numeric inputs are stored as floats, so 9 becomes 9.0. An invalid type, out-of-range value, nonfinite value or unsupported method raises `PlanningModelError` during construction.

Run after TODO 1 – repository root

```
python -m pytest tests/public/test_settings.py -q
```

Expected output (summary)

```
36 passed
```

This focused check covers settings only. Later model tests are still expected to fail until their TODOs are complete.

TODO 2 – Connect objects into one valid problem

Edit: ParametricPlanningProblem.__post_init__. Apply these construction checks:

- Check types before reading properties: `start` and `goal` must be `Configuration`, `bounds` must be `ConfigurationBounds`, `obstacle` must be `Obstacle`, `path` must be `Path`, and `settings` must be `PlanningSettings`.
- Require two-dimensional start, goal, bounds and every path waypoint.
- Require 2–32 path waypoints, including the endpoints.
- Match the path endpoints to start and goal using object equality.
- Check every waypoint with `bounds.contains(point)`. The workspace boundary is allowed.
- Reject equal consecutive positions. Such a segment has zero length and cannot supply a direction for the turning calculation. A closed loop whose consecutive positions differ is allowed.

Raise `PlanningModelError` for an inconsistent request. Do not repair its coordinates or replace its endpoints.

Expected: a request whose objects satisfy these checks can be constructed. An inconsistent request raises `PlanningModelError`. Construction does not perform a collision calculation.

Run after TODO 2 – repository root

```
python -m pytest tests/public/test_problem.py -q
```

Expected output (summary)

```
20 passed
```

The supplied obstacle constructor already checks its 2-D centre and positive radius. Check the obstacle type, not a nonexistent `obstacle.dimension`. Straight segments between points inside this rectangular workspace stay inside it. The supplied calculation functions already reject nonfinite results and requests above 10,000 samples. These are not additional TODOs.

TODO 3 – Return the evaluated response

Edit: ParametricPlanningProblem.evaluate. Use the three completed functions from `src/ap_week05/provided.py`. First, give `sample_path` the stored path, resolution and method, and receive the array of positions. Give this array, the original path and the obstacle to `path_metrics`. Finally, give the returned length and clearance and the two limits to `constraint_margins`. Section 4.2 connects these inputs and returned values to the existing `examples/inspect_path.py`.

Construct the supplied `EvaluationResult` using its named fields from Section 3.2. Preserve raw measurements and signed margins. Set `feasible` to a Python Boolean that is true precisely when both margins are nonnegative. Include the selected method and the number of sampled rows. Return the object. Do not print it, save a file or call the renderer inside `evaluate`. The provided demo is responsible for printing.

Expected: return an `EvaluationResult` containing measurements, signed margins and the decision. In interpolated mode, the direct path returns a normal result with a negative clearance margin. It must not be rejected merely because it intersects the disk. Invalid settings and an evaluated constraint failure have different outcomes.

Run after TODO 3 – repository root

```
python -m pytest tests/public/test_evaluation.py -q
```

Expected output (summary)

```
9 passed
```

After TODOs 1–3, the supplied demo in `src/ap_week05/__main__.py` can use your completed model. The command `python -m ap_week05` runs that file, which receives the result and prints its fields. The `to_plot_data()` method is already complete and stays unchanged. The full test command below also checks its behavior with your completed model.

5.4 Write tests, run the final checks and read the results

TODO 4 – Test three different behaviors

Edit: `tests/test_student.py`. Replace the skipped placeholder with your own tests. Give each test an informative name beginning with `test_`. Cover each of the following behaviors with an assertion about the result, not merely a call that finishes without an error.

1. **Feasible candidate comparison.** Evaluate the direct path and detour with the same default checking conditions from Section 4.1. Check the expected lengths (4 and 8 m), clearances (−0.5 and 1.5 m), margins and decisions. The detour must be feasible, while the direct path fails clearance.
2. **An inclusive boundary.** Reuse the saved Section 4.2 detour with minimum clearance 0.5 m. Compare maximum length 8.0 with 7.999 m. Check unchanged measurements, length margins 0 and −0.001 m, and `True` then `False`. Only the acceptance limit changes.
3. **An infeasible result exposed by checking.** Keep the direct path, scene, resolution 0.5 m and default limits fixed. Compare `waypoints` with `interpolated`. Check the sample counts (2 and 9), unchanged length, clearances (1.5 and −0.5 m), clearance margins (1 and −1 m) and `True` then `False`. The different decision must come from checking different positions on the same path.

These are three behavior checks, not a request to reproduce every provided test. Do not copy or import public test functions. Use small inputs whose expected values you can calculate directly. An expected value obtained by using the same calculation function as your implementation does not independently check that calculation.

For a floating-point measurement, an assertion such as the following compares against a stated tolerance. Here `actual` is the result you obtained and `expected` is your independently calculated value.

```
assert actual == pytest.approx(expected, abs=1e-12)
```

Near zero, state an absolute tolerance. Compare Boolean outcomes directly. The supplied public tests already cover invalid inputs and data independence. Your three tests focus on the evaluation decision and the numbers that explain it.

Run the full suite after completing your model and three behavior checks.

Run – repository root

```
python -m pytest tests/public tests/test_student.py -q
```

Completed reference example (summary)

```
74 passed
```

This reference example has 71 public tests and three independent tests. Your total may differ if a test is parametrized or you add tests. All three required behaviors must be covered, with no failure or skipped placeholder.

Run – repository root

```
python -m ap_week05
```

Expected output

```
length_m: 8.000000
clearance_m: 1.500000
total_turn_rad: 3.141593
clearance_margin_m: 1.000000
length_margin_m: 1.000000
feasible: True
method: interpolated
sample_count: 17
```

The demo prints the data returned by your model. Passing tests confirms the cases run, not every possible input. GitHub/LMS submission is the next step.

5.5 Push the code and submit through LMS

Before submission, confirm each result separately:

- TODOs 1–3 are completed in `src/ap_week05/model.py`.
- `tests/test_student.py` contains the three required behavior checks.
- The complete required test command passes without the skipped placeholder.
- `python -m ap_week05` produces the stated detour result.
- The supplied files and interfaces remain unchanged.

GitHub – review and upload the two edited files

Run from the repository root in your own private repository. `add` selects files, `commit` records the change locally, and `push` uploads it. Run the commands in order and review each diff before continuing. Only the intended model and test changes should be staged. Do not add environment directories, caches, optional outputs or credentials. The example commit message may be replaced with a meaningful description.

```
git diff
git status --short
git add src/ap_week05/model.py tests/test_student.py
git diff --staged
git commit -m "Complete Week 5 parametric evaluation"
git push origin main
git status --short
git rev-parse HEAD
git ls-remote origin refs/heads/main
```

Press `q` to leave a diff pager. The status after pushing should be clean. Compare the full local commit ID with the first field of the remote result. They must match. If they differ, the remote does not yet identify the same work as your local commit. Keep the repository private.

LMS – submit the same three fields used in Weeks 1–4

At LMS submission, open repository **Settings**, then **Collaborators** or **Manage access**, and invite the exact account SSUMechE. Check the account spelling. Report whether the invitation is pending or accepted. A correct invitation sent on time and awaiting only instructor acceptance is not a student omission. Submit these three fields:

1. Your own private Week 5 repository URL.
2. The full 40-character commit ID of the pushed revision to be graded.
3. Confirmation of private visibility, the actual SSUMechE access state (pending or active), and that the recorded commit is the grading revision.

Due: within one week of the corresponding lab. Use the exact deadline in the LMS notice. GitHub upload alone is not submission.

Fictional format example – replace every example value

```
Private repository URL:
https://github.com/example-student/applied-programming-w05-20260000
Full commit ID:
0123456789abcdef0123456789abcdef01234567
Confirmation:
Private. SSUMechE invitation sent on time and pending acceptance.
Grade the full commit ID recorded above.
```

The username, student ID and commit above are fictional examples. If access is accepted, report active access instead of pending. Do not submit the public template URL, a local folder path, a branch name, **latest**, a short hash, an unpushed commit or any credentials. If a correctly sent invitation remains pending after the announced checking time, contact the instructor through the stated LMS procedure.

Missing mandatory submission-format requirements by the deadline invokes the announced 0-point submission rule, with the timely pending-invitation exception above. This rule does not mean that every failed numerical test automatically gives zero for the entire assignment. Later pushes do not silently replace the commit recorded in LMS. Week 6 will reuse this evaluator to separate and test component responsibilities. That continuation does not postpone the Week 5 submission.

5.6 Optional path views and 3D replay

Use each optional view after the TODO checkpoint stated below. These views are not an extra stage required after submission. Skip these views if you do not want or cannot display them. The tests and submission conditions remain the same.

Optional: view the path and checked points as a picture

After TODOs 1 and 2, run this from the repository root:

```
python scripts/render_preview.py
```

Generated file

```
artifacts/path_preview.svg
```

Open this file in a browser to see the path, sampled points and disk. It is not required for tests or submission. If it cannot be displayed, use the numerical result above. No graphics package installation is needed.

Optional: compare the same path in 3D – not submitted

After completing model TODOs 1–3, you may run the supplied renderer to see *where* your program checked the path. This visualizes the same CPU method comparison from Section 2.2 using your completed evaluator. There is no new

model or rendering code to write, and no movie or extra report to submit. You may skip this activity and use Section 5.5 for submission. Required tests and submission do not need a GPU.

The renderer calls your `evaluate()` and the provided `to_plot_data()`. It displays the returned decision beside a replay of the same path. The map remains two-dimensional. A low cylinder shows the disk's footprint, small fixed dots show sampled positions, and a moving marker follows the path. The cylinder height and marker size are only for display. The marker is not a robot with a collision radius. This Genesis-based **geometric replay** does not simulate a controller, forces or contact, and does not verify safety.

Use the CUDA environment already prepared

First complete the separate *CUDA and Genesis GPU setup guide* and its CUDA and Genesis checks. Use that existing `applied-programming-cuda` environment. Do not create another one for this preview. In Anaconda Prompt, return to the same Week 5 repository root and run:

```
set PYTHONUTF8=1
conda activate applied-programming-cuda
python -m pip install -e . --no-deps --no-build-isolation
```

This installs the current local Week 5 package without replacing the GPU environment's dependencies. Do not install Week 4's CPU packages or rerun Week 5's core requirements in this CUDA environment. If the earlier GPU checks have not passed, use the numerical result or SVG instead.

Run – repository root, CUDA environment

```
python scripts/preview_genesis.py --case direct --method waypoints
```

The input is Figure 5.1(b)'s direct path, (0, 0) to (4, 0). The scene, clearance limit 0.5 m and length limit 9 m are unchanged. With a correct model, the result has two checked points, clearance 1.5 m and `feasible=True`. The replay still passes through the disk between those points. A passing sampled check did not make that segment safe.

Now keep the path and limits fixed and change only the checking method:

```
python scripts/preview_genesis.py --case direct --method interpolated
```

At the default 0.5 m resolution, the result has nine checked points. One is at the disk centre, giving clearance -0.5 m, clearance margin -1.0 m and `feasible=False`. The path is unchanged. More positions were inspected. These expected values explain the comparison. The renderer shows your actual returned result, so an incorrect implementation is not replaced with these values.

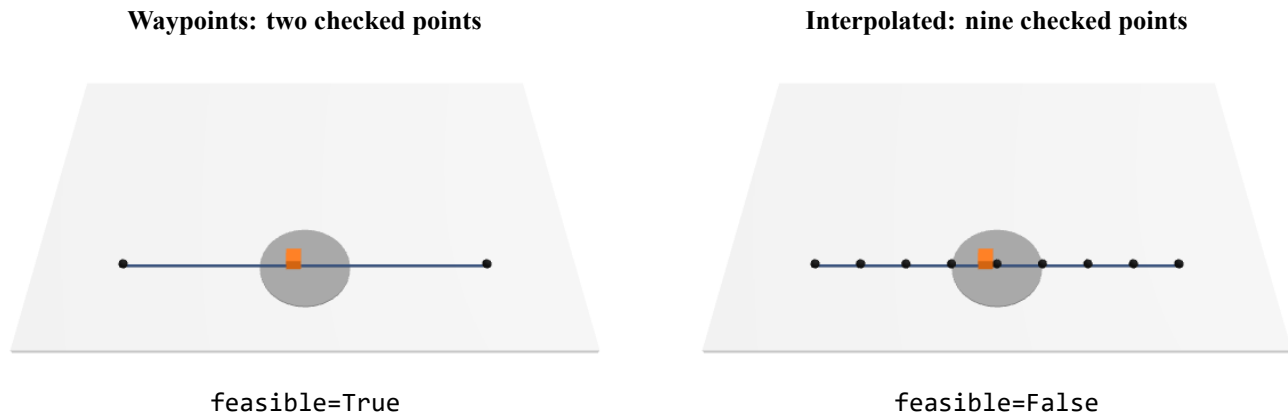


Figure 5.2: Actual Genesis-rendered frames of the completed reference model's same direct path, viewed from the same camera. Black dots are checked positions. The orange cube marks the moving point and is not an extra check. Only the right-hand evaluation checks the disk's interior. The replay is not a contact simulation.

Open and interpret the result

Each command prints the location of its generated `index.html` under `artifacts/genesis/`. Open that file in a browser. The page shows the actual numerical result and, if rendering succeeded, a playable `preview.mp4`. The two default runs save under `direct_waypoints_L9_low` and `direct_interpolated_L9_low`, respectively, so they can be compared. All these files are ignored local outputs, not submissions.

The default `low` profile uses one small scene and a four-second video at 640×480 pixels and 10 frames per second. It is intended for trying this activity on a 6 GB GPU, not a guarantee that every such PC will run it. First execution may take longer while code compiles. CUDA computation and graphics output are separate checks. A CUDA-check pass does not prove that the graphics driver can render this preview. If rendering fails or resources are insufficient, stop the preview and use the saved numerical result or the SVG. Do not change security settings or reinstall the driver merely to complete this optional activity.

Return to the required Week 5 environment

```
conda activate applied-programming-w05
python -m pytest tests/public tests/test_student.py -q
```

Use Section 5.5 for submission. The optional comparison does not replace your own tests or add anything to the LMS submission.